

TEACH YOURSELF LECTURE

Lecture 03

Programming Fundamentals

Chapter lecture — 18 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

Computer Science XII · Bilingual support: English + اردو

کمپیوٹر سائنس — بارہویں جماعت: خود سیکھیں، خود پرکھیں

18 teach-yourself lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this lecture

Learn this chapter **on your own**. Read *Learn it*, try the worked example on paper, then answer *Check yourself* before looking at the answers. The green box repeats the idea in Urdu. ★ Golden topics are high-yield in board exams.

یہ لیکچر خود سیکھیں۔ مثال کاغذ پر حل کریں، پھر جواب دیکھنے سے پہلے خود کو پرکھیں۔

Lectures in this pack

Chapter 3: Programming Fundamentals. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 3.1 Data Structures in Python

L03. 3.1.2 Tuples (Ordered, Immutable)

L05. 3.1.4 Dictionaries (Key-Value Pairs)

L07. 3.1.6 Activity: Student Attendance System

L09. 3.2 Functions in Python

L11. 3.2.2 Functions with Return Values

L13. 3.2.4 Scope of Local and Global Variables

L15. 3.6 The with Statement (Context Manager)

L17. 3.9 Mini File Handling Project: Grade Tracker

L02. 3.1.1 Lists (Ordered, Mutable, Methods)

L04. 3.1.3 Sets (Unordered, Unique, Math Operations)

L06. 3.1.5 Common Built-in Functions (min, max, len, sum)

L08. 3.1.7 Activity: Word Frequency Counter

L10. 3.2.1 Types of Functions (Built-in vs User-defined)

L12. 3.2.3 Activity: Arithmetic Calculator Using Functions

L14. 3.3 & 3.4 File Handling and Operations (Modes r, w, a)

L16. 3.8 Errors and Exceptions (Try-Except)

L18. Career Opportunities

CHAPTER 3

Programming Fundamentals

This chapter teaches you how Python stores collections of data, how you write reusable functions, and how you save information in files so it is still there after the program ends. You will build small programs — attendance, a word counter, a calculator, and a grade tracker — that you can type and run yourself.

اس باب میں آپ سیکھیں گے کہ پائتھن (Python) ڈیٹا کو فہرست، ٹیپل، سیٹ اور ڈکشنری میں کیسے رکھتا ہے، فنکشن کیسے لکھے جاتے ہیں، اور فائلوں میں ڈیٹا کیسے محفوظ کیا جاتا ہے تاکہ پروگرام بند ہونے کے بعد بھی وہ موجود رہے۔

What you will learn

- Distinguish primitive types from abstract data structures (list, tuple, set, dictionary).
- Use tuples as ordered, immutable collections, including indexing and the two tuple methods.
- Store and retrieve key-value data with dictionaries, including safe access with `.get()`.
- Write user-defined functions with parameters and `return` values, and contrast them with built-in functions.
- Open text files in modes `r`, `w` and `a`, and close them safely with `with`.
- Create, index, slice, traverse and update lists using the main list methods.
- Build sets of unique items and compute union, intersection, difference and symmetric difference.
- Apply `min()`, `max()`, `len()`, `sum()` and `sorted()` across data structures.
- Explain local and global variable scope with short programs.
- Catch file errors with `try / except` and complete a grade-tracker file project.

3.1 Data Structures in Python

LEARN IT

A **data structure** is a way of organising values in a program so you can store them, find them and change them. Every useful program is built on data structures. Python gives you two layers:

- **Primitive data structures** — the basic types that hold **one** value. Examples: integers (`int`), floating-point numbers (`float`), and booleans (`bool`).
- **Abstract data structures** — higher-level containers built from primitive types so you can hold **many** values and run richer operations. Examples: **lists, tuples, sets, dictionaries**.

Python has these four abstract structures built in. You do not install anything extra. That is why collecting marks, names or settings is simpler in Python than in many other languages.

Category	Description	Examples
Primitive	Basic types that represent a single value.	<code>int</code> , <code>float</code> , <code>bool</code>
Abstract	Higher-level structures for collections and complex operations.	Lists, tuples, sets, dictionaries

WORKED EXAMPLE

One value versus a collection. A student's age is one integer. That student's three test marks need a list.

```
age = 17
height = 5.8
passed = True
marks = [80, 75, 90]
print(type(age))
print(type(height))
print(type(passed))
print(type(marks))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
```

`type()` tells you which structure a value has. `age` is a primitive integer. `marks` is an abstract list that holds three integers together.

REMEMBER IT

Primitive = one piece. Abstract = a box of pieces. Think of a single rupee coin (primitive) versus a wallet that can hold many coins (list, tuple, set or dictionary).

COMMON MISTAKE

Do not call a list a “variable type” and stop there. A list is a **structure**: it has an order, methods, and rules. The four abstract structures look similar at first (they all hold several values) but they follow different rules. Learn those rules one by one in 3.1.1–3.1.4.

ڈیٹا سٹرکچر (Data Structure) پروگرام میں ڈیٹا کو ترتیب دینے کا طریقہ ہے۔ پرائمیٹو اقسام جیسے عدد (int) صرف ایک قیمت رکھتی ہیں، جبکہ فہرست، ٹیپل، سیٹ اور ڈکشنری کئی قیمتیں ایک ساتھ سنبھالتی ہیں۔ پائتھن میں یہ چاروں سٹرکچرز پہلے سے موجود ہیں، اس لیے ڈیٹا جمع کرنا دوسری زبانوں کے مقابلے میں آسان ہے۔

CHECK YOURSELF

1. Name two primitive types and two abstract data structures in Python.
2. Why is a list called an abstract data structure rather than a primitive type?

Answers: 1. Primitive: any two of int, float, bool. Abstract: any two of list, tuple, set, dictionary. 2. Because it is a higher-level container built from primitive values, holding many items and supporting operations such as append and slice.

3.1.1 Lists (Ordered, Mutable, Methods) ★ Golden

LEARN IT

A **list** is Python's most versatile built-in data structure. It stores an **ordered** collection of items in a single variable. Lists are **mutable** (you can change them after you create them) and they are written with **square brackets** `[]`.

Why use a list? One variable can hold thousands of values. You can mix types if you need to. You can add, remove and sort items as the program runs.

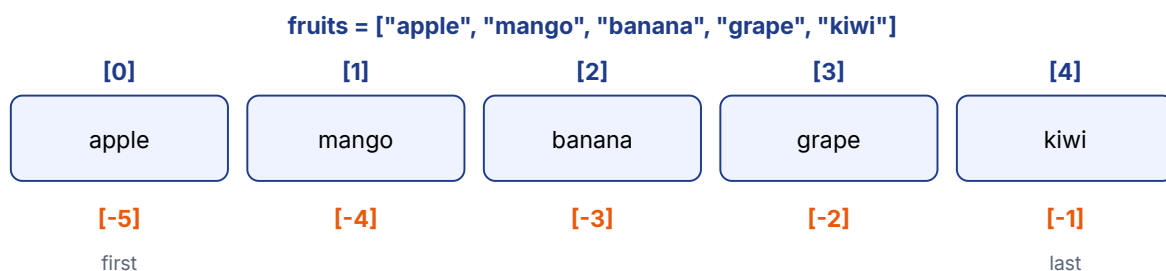
```
numbers = [10, 2, 15, 14, 3, 33]      # list of integers
names = ["Hassan", "Qasim", "Malika"] # list of strings
mixed = [1, "Python", 3.14, True]    # mixed data types
print(numbers)
print(names)
print(mixed)
```

OUTPUT

```
[10, 2, 15, 14, 3, 33]
['Hassan', 'Qasim', 'Malika']
[1, 'Python', 3.14, True]
```

List indexing (positive and negative)

Each item sits in a numbered slot called an **index**. Numbering starts at **0** from the left. Python also supports **negative indexing** from the right: **-1** is the last item, **-2** is the second last, and so on.



Positive indices count left to right from 0. Negative indices count right to left from -1.

```
fruits = ["apple", "mango", "banana", "grape", "kiwi"]
print(fruits[0])    # first item
print(fruits[4])    # last item (when you know the length)
print(fruits[-1])   # last item (works even if the list grows)
print(fruits[-5])   # first item, counted from the right
```

OUTPUT

```
apple
kiwi
kiwi
apple
```

REMEMBER IT

Always remember `[-1]`. If a list has 10,000 items and you do not want to compute `len(list) - 1`, write `list[-1]`. It grabs the last item in one step.

Slicing a list

Slicing copies a portion of a list into a **new** list. The original is not changed.

Syntax: `list[start:stop:step]`

- **start** — first index to include (default 0).
- **stop** — first index to **exclude**. The slice goes up to, but does **not** include, this index.
- **step** — jump size (default 1). A step of `-1` walks backwards.

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[2:6])
print(nums[:4])
print(nums[6:])
print(nums[:2])
print(nums[::-1])
```

OUTPUT

```
[2, 3, 4, 5]
[0, 1, 2, 3]
[6, 7, 8, 9]
[0, 2, 4, 6, 8]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Trace of `nums[2:6]`: start at index 2 (value 2), take 3, 4, 5, and stop **before** index 6. Result `[2, 3, 4, 5]`. `nums[::-1]` is the standard way to reverse a list without changing the original (it returns a new reversed list).

Traversal

Traversal means visiting each element, one by one, in order. Use a `for` loop (the usual Python style) or a `while` loop with an index.

```
numsList = [10, 20, 30]
for num in numsList:
    print(num)
```

OUTPUT

```
10
20
30
```

```
numsList = [10, 20, 30]
index = 0
while index < len(numsList):
    print(numsList[index])
    index += 1
```

OUTPUT

```
10
20
30
```

The `for` loop gives you each value directly. The `while` loop needs a counter, a test `index < len(numsList)`, and `index += 1` so the loop ends.

Built-in list methods

Because a list is mutable, Python gives you methods that change it in place. `len()` is listed with them in many notes, but it is a **built-in function**: you write `len(fruits)`, not `fruits.len()`.

Operation	What it does
<code>len(list)</code>	Returns the number of items.
<code>.append(value)</code>	Adds an item at the end .
<code>.insert(index, value)</code>	Inserts an item at a chosen position (later items shift right).
<code>.remove(value)</code>	Removes the first matching value. Raises <code>ValueError</code> if it is missing.
<code>.clear()</code>	Removes every item. The list becomes <code>[]</code> .
<code>.count(value)</code>	How many times a value appears.
<code>.index(value)</code>	Index of the first occurrence.
<code>.sort()</code>	Sorts the list in place, ascending. Use <code>reverse=True</code> for descending.
<code>.reverse()</code>	Reverses the list in place (does not sort).

WORKED EXAMPLE

Worked example: methods change the same list step by step.

```
fruits = ['apple', 'banana', 'apple']
print(fruits)
fruits.append('grape')
print(fruits)
fruits.remove('apple')
print(fruits)
fruits.sort()
print(fruits)
```

OUTPUT

```
['apple', 'banana', 'apple']
['apple', 'banana', 'apple', 'grape']
['banana', 'apple', 'grape']
['apple', 'banana', 'grape']
```

After this line	List in memory
create	<code>['apple', 'banana', 'apple']</code>
<code>.append('grape')</code>	<code>['apple', 'banana', 'apple', 'grape']</code>
<code>.remove('apple')</code>	<code>['banana', 'apple', 'grape']</code> — only the first apple goes
<code>.sort()</code>	<code>['apple', 'banana', 'grape']</code>

More methods on numbers:

```
nums = [10, 20, 30]
print(len(nums))
nums.insert(1, 15)
print(nums)
print(nums.count(20))
print(nums.index(30))
nums.reverse()
print(nums)
nums.clear()
print(nums)
```

OUTPUT

```
3
[10, 15, 20, 30]
1
3
[30, 20, 15, 10]
[]
```

`insert(1, 15)` places 15 at index 1, so 20 and 30 shift right. After that, 30 sits at index 3. `reverse()` flips the current order; it does not sort. `clear()` leaves an empty list `[]`, not the value `None`.

You can also replace an item by index, because lists are mutable:

```
numbers = [10, 2, 15]
numbers[0] = 99
print(numbers)
```

OUTPUT

```
[99, 2, 15]
```

Advantages of lists

- **Mutable** — add, remove or replace items as the program runs.
- **Organised** — thousands of values live in one variable instead of `n1`, `n2`, `n3`...
- **Flexible** — integers, strings, booleans and even other lists can sit in the same list.

Activity: searching an element in a list

This program combines a list, user input, a `for` loop, an `if` test and a **flag** (a Boolean that remembers whether the search succeeded).

```
numbers = [5, 10, 15, 20, 25]
search = int(input("Enter number to search: "))
found = False
for num in numbers:
    if num == search:
        found = True
        break
if found:
    print("Number found in the list.")
else:
    print("Number NOT found in the list.")
```

Execution 1 — the user types 15, which is at index 2. The loop sets `found = True` and `break` stops the scan at once.

OUTPUT

```
Enter number to search: 15
Number found in the list.
```

Execution 2 — the user types 99. The loop finishes without a match, so the flag stays `False`.

OUTPUT

```
Enter number to search: 99
Number NOT found in the list.
```

Why `break`? If the value is found at index 1 in a list of 10,000 items, there is no need to check the remaining 9,998. `break` makes linear search cheaper as soon as it succeeds.

REMEMBER IT

LIST — Lots of items, Indexed from 0, Square brackets, Tweakable (mutable). Slice memory trick: “start is in, stop is out.”

COMMON MISTAKE

Mistake 1: `.remove('apple')` deletes every apple. **Fix:** it deletes only the first match. Call it again if you need the next one, or use a loop.

Mistake 2: `.sort()` and `sorted()` are the same. **Fix:** `.sort()` is a list method that changes the original and returns `None`. `sorted()` is a function that works on many structures and returns a **new** list (see 3.1.5).

Mistake 3: `fruits[5]` on a five-item list. Valid positive indices are 0 to 4. Use `[-1]` for the last item.

اردو میں سمجھیں

فہرست (List) مربع بریکٹ `[]` میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable)۔ انڈیکس صفر سے شروع ہوتا ہے، اور `[-1]` آخری عنصر دیتا ہے۔ `.append()` آخر میں جوڑتا ہے، `.remove()` پہلی مشابہ قیمت ہٹاتا ہے، اور `.sort()` اصل فہرست کو ترتیب دیتا ہے۔ تلاش کرتے وقت `break` مل جانے کے بعد لوپ روک دیتا ہے۔

EXAM TIP

Lists appear every year: “What is a list?”, “List four methods with one line each”, “Differentiate list and tuple”, “Write a program to search an element”, “Predict the output of a slice.” Write: ordered, mutable, `[]`, index from 0, `[-1]` for last. For a slice, always say the stop index is excluded. For `.remove()`, say “first occurrence only.” Keep a short program ready that uses `append`, `remove` and `sort` with the printed list after each line.

CHECK YOURSELF

1. What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
2. After `fruits = ['apple', 'banana', 'apple']` then `fruits.remove('apple')`, what is `fruits`?
3. Why does the search program use `break`?

Answers: 1. `[2, 3, 4, 5]`. 2. `['banana', 'apple']` (only the first apple is removed). 3. To stop scanning once the value is found, so the rest of a long list is not checked.

3.1.2 Tuples (Ordered, Immutable) ★ Golden

LEARN IT

A **tuple** is an ordered collection that is **immutable** — once created, you cannot add, remove or replace items. Think of it as a **read-only list**. Tuples use **parentheses** `()`. Like lists, they allow duplicates, mixed types, and indexing from 0 (including negative indices).

List []

Can add / remove / replace

Tuple ()

LOCKED — read-only

A list is an open box you can refill. A tuple is a sealed box you may only look into.

```
t = (10, 20, 30, 40, 50)
print(t[0])
print(t[-1])
print(t[1:4])
print(t[::-1])
```

OUTPUT

```
10
50
(20, 30, 40)
(50, 40, 30, 20, 10)
```

Slicing a tuple returns a **new tuple**, not a list. Negative indexing and `[::-1]` work exactly as they do for lists.

Immutability

You cannot assign to an index of a tuple. Python raises `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```

OUTPUT

```
Traceback (most recent call last):
  File "tuple.py", line 2, in <module>
    t[0] = 99
TypeError: 'tuple' object does not support item assignment
```

Exception: if a tuple contains a **mutable object** (a list inside the tuple), that inner object's contents can still change. The tuple still holds the same two list objects; you changed a list, not the tuple's structure.

```
t = ([1, 2], [3, 4])
t[0].append(99)
print(t)
```

OUTPUT

```
([1, 2, 99], [3, 4])
```

`t[0] = [9, 9]` would still be illegal, because that would replace the tuple's first item. `t[0].append(99)` is legal because it mutates the list that already lives at index 0.

Traversal and methods

Traverse a tuple with `for`, just as you do a list. You are only reading, which is the usual reason to choose a tuple.

```
student = ("Ali", 20, "Computer Science")
for item in student:
    print(item)
```

OUTPUT

```
Ali
20
Computer Science
```

Because a tuple cannot grow or shrink, it has only **two** methods:

Method	Meaning
<code>.count(value)</code>	How many times the value appears.
<code>.index(value)</code>	Index of the first occurrence.

```
t = (10, 20, 10, 30, 10)
print(t.count(10))
print(t.index(20))
```

OUTPUT

```
3
1
```

Advantages of tuples

- **Protection** — coordinates, identity-card numbers or settings cannot be changed by accident.
- **Speed** — a fixed-size tuple is processed slightly faster than a list.
- **Hashable keys** — a tuple of immutables can be used as a dictionary key; a list cannot. (You will use dictionaries in 3.1.4.)

WORKED EXAMPLE

When to choose a tuple. A student's roll number, name and class form a record that should not be edited in place:

```
record = ("XII-CS-07", "Ayesha", "Computer Science")
print(record[0])
print(len(record))
```

OUTPUT

```
XII-CS-07
3
```

If later you need to change the class, you build a **new** tuple. You do not assign to `record[2]`.

REMEMBER IT

Tuple = locked lunchbox. You can see what is inside; you cannot swap the sandwich. Memory: "Tuple is Tight (immutable), written with round brackets that look like a hug holding the data still."

COMMON MISTAKE

One-item trap: `(5)` is just the integer 5 with extra parentheses. A one-item tuple needs a trailing comma: `(5,)`.

```
t1 = (5)
t2 = (5,)
print(type(t1), t1)
print(type(t2), t2)
```

OUTPUT

```
<class 'int'> 5
<class 'tuple'> (5,)
```

An empty tuple is `()`.

اردو میں سمجھیں

ٹیپل (Tuple) گول بریکٹ `()` میں لکھا جاتا ہے، ترتیب رکھتا ہے مگر تبدیل نہیں ہو سکتا (Immutable). انڈیکس اور سلائس فہرست جیسی ہیں، لیکن `t[0] = 99` سے `TypeError` آتا ہے۔ صرف دو میتھڈز ہیں: `count()` اور `index()`۔ حساس ڈیٹا کے لیے ٹیپل محفوظ ہے۔ اگر ٹیپل کے اندر فہرست ہو تو اس فہرست کے اندر کی اشیاء بدل سکتی ہیں۔

EXAM TIP

Expect “Differentiate list and tuple” (at least four points: brackets, mutability, methods, speed/protection), “Why use a tuple?”, and “What happens if you write `t[0] = 1`?” Quote the `TypeError` wording. Mention the inner-list exception if the question is a 6-mark “explain with example.” A one-item tuple `(5,)` is a favourite trick question.

CHECK YOURSELF

1. Name the two tuple methods.
2. What error does `t[0] = 99` raise for a tuple?
3. After `t = ([1, 2], [3, 4]); t[0].append(99)`, what is `t`?

Answers: 1. `count` and `index`. 2. `TypeError: 'tuple' object does not support item assignment`. 3. `([1, 2, 99], [3, 4])`.

3.1.3 Sets (Unordered, Unique, Math Operations)

LEARN IT

A **set** stores an **unordered, unindexed** collection of **unique** elements. It is inspired by mathematical sets. You write a set with **curly braces** `{}`. Duplicates are removed as soon as the set is created. Because there is no index, `myset[0]` is illegal.

The set itself is **mutable** (you can add and remove items). Each item inside must be **immutable** (numbers, strings, tuples of immutables — not lists).

```
nums = {1, 4, 3, 3, 2, 2}
print(nums)
```

OUTPUT

```
{1, 2, 3, 4}
```

The two 3s and two 2s collapse to one each. Do not depend on the printed order: a set is unordered. CPython often prints small integers in a tidy way, but that is not a language promise.

Adding and removing

Method	Description
<code>.add(value)</code>	Adds one element.
<code>.update(values)</code>	Adds many elements from a list, tuple or set.
<code>.remove(value)</code>	Removes the value. Raises <code>KeyError</code> if it is missing.
<code>.discard(value)</code>	Removes the value. Does not crash if it is missing.
<code>.clear()</code>	Removes every element. Result is <code>set()</code> .

```
s = {1, 2, 3}
s.add(4)
print(s)
s.update([5, 6])
print(s)
s.discard(99)
print(s)
s.remove(1)
print(s)
s.clear()
print(s)
```

OUTPUT

```
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
{2, 3, 4, 5, 6}
set()
```

`discard(99)` quietly does nothing. After `clear()` Python prints `set()`, not `{}`, because `{}` already means an empty **dictionary**.

Compare with `.remove(99)` on a set that does not contain 99:

```
s = {1, 2, 3}
s.remove(99)
```

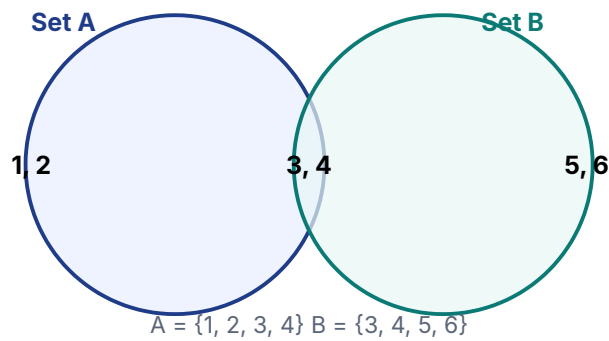
OUTPUT

```
Traceback (most recent call last):
  File "set_remove.py", line 2, in <module>
    s.remove(99)
KeyError: 99
```

Prefer `.discard()` when the item might not be there, so a missing value does not crash the program.

Mathematical operations

Sets are built for union, intersection, difference and symmetric difference.



Overlap {3, 4} is the intersection. Left-only {1, 2} is $A - B$. Right-only {5, 6} is $B - A$.

Operation	Meaning	Syntax	Result for A, B above
Union	All unique items from both	$A \mid B$ or <code>A.union(B)</code>	{1, 2, 3, 4, 5, 6}
Intersection	Items in both	$A \& B$ or <code>A.intersection(B)</code>	{3, 4}
Difference	In A, not in B	$A - B$ or <code>A.difference(B)</code>	{1, 2}
Symmetric difference	In either, but not both	$A \wedge B$ or <code>A.symmetric_difference(B)</code>	{1, 2, 5, 6}

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)
print(A & B)
print(A - B)
print(A ^ B)
```

OUTPUT

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

$B - A$ would be {5, 6} — difference is not symmetric. Symmetric difference $A \wedge B$ is $(A - B) \mid (B - A)$.

Set traversal

There is no index, so you loop with `for`. Print order is not guaranteed. Use `sorted()` when you need a stable alphabetical or numerical order (it returns a list).

```
colors = {"red", "green", "blue"}
for color in sorted(colors):
    print(color)
```

OUTPUT

```
blue
green
red
```

WORKED EXAMPLE

Deduplication. A teacher types roll numbers and some are repeated. A set keeps each roll once:

```
rolls = [12, 5, 12, 8, 5, 19]
unique = set(rolls)
print(unique)
print(sorted(unique))
```

OUTPUT

```
{8, 19, 12, 5}
[5, 8, 12, 19]
```

`set(rolls)` drops duplicates. `sorted(unique)` gives a list in numeric order for a register.

REMEMBER IT

SET = Same-item Eaten Twice-never. Empty set is `set()`, never `{}`. Safe delete = `discard`; angry delete = `remove`.

COMMON MISTAKE

`myset[0]` raises `TypeError` because sets are unindexed. Convert with `list(myset)` only if you accept that the order is arbitrary. Do not put a list inside a set — lists are mutable, so they cannot be set elements.

اردو میں سمجھیں

سیٹ (Set) منفرد عناصر کا بغیر ترتیب اور بغیر انڈیکس مجموعہ ہے، جو `{}` میں لکھا جاتا ہے۔ دہرائی ہوئی قیمتیں خود بخود ختم ہو جاتی ہیں۔ `add()` اور `update()` عناصر جوڑتے ہیں، `discard()` محفوظ طریقے سے ہٹاتا ہے جبکہ `remove()` غیر موجود عنصر پر `KeyError` دیتا ہے۔ اجتماع (Union) `|`، اشتراک (Intersection) `&`، فرق - اور متوازی فرق `^` ریاضی کی کارروائیاں ہیں۔

CHECK YOURSELF

1. What is printed by `print({1, 4, 3, 3, 2, 2})` ?
2. Why prefer `.discard()` over `.remove()` ?
3. If $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$, what is $A \cap B$?

Answers: 1. `{1, 2, 3, 4}` (duplicates removed; order may vary). 2. `discard` does not raise `KeyError` if the value is missing. 3. `{3, 4}`.

3.1.4 Dictionaries (Key-Value Pairs) ★ Golden

LEARN IT

A **dictionary** stores data as **key-value pairs**. It works like a real dictionary: you look up a word (the **key**) to find its meaning (the **value**). Write a dictionary with curly braces, and put a **colon** between each key and value.

- **Ordered** (Python 3.7+): insertion order is kept.
- **Mutable**: you can add, change and delete pairs.
- **Keys are unique**: if you repeat a key, the last value silently overwrites the earlier one. Values may repeat.



You never search a dictionary by position. You search by key.

A school analogue: a **roll number** (key) can point to only one student (value). If you assign that roll number to someone else, the old link is gone.

Adding, changing and deleting

```
d = {"name": "Alice", "age": 30}
print(d)
d["age"] = 31
print(d)
d["city"] = "NYC"
print(d)
del d["city"]
print(d)
```

OUTPUT

```
{'name': 'Alice', 'age': 30}
{'name': 'Alice', 'age': 31}
{'name': 'Alice', 'age': 31, 'city': 'NYC'}
{'name': 'Alice', 'age': 31}
```

Assigning to an existing key updates the value. Assigning to a new key adds a pair. `del d["city"]` removes that pair.

The duplicate-keys rule. Python does not crash. The last assignment wins.

```
d = {"a": 1, "a": 2, "a": 3}
print(d)
```

OUTPUT

```
{'a': 3}
```

Accessing values safely

1. Direct brackets (risky)

`dict[key]` works if the key exists. If it is missing, the program crashes with `KeyError`.

2. `.get()` (safe)

`dict.get(key)` returns `None` if the key is missing. You may pass a default: `dict.get(key, 0)`.

```
person = {"name": "Alice", "age": 30}
print(person["name"])
print(person.get("salary"))
print(person.get("salary", 0))
print("name" in person)
print("salary" in person)
```

OUTPUT

```
Alice
None
0
True
False
```

The operator `in` tests **keys**, not values. `"Alice" in person` would be `False`.

```
person = {"name": "Alice", "age": 30}
print(person["salary"])
```

OUTPUT

```
Traceback (most recent call last):
  File "keyerror.py", line 2, in <module>
    print(person["salary"])
KeyError: 'salary'
```

Dictionary views and traversal

Three view methods return live objects. If the dictionary changes, the view reflects the change immediately.

Method	Returns
<code>.keys()</code>	All keys.
<code>.values()</code>	All values.
<code>.items()</code>	Key–value pairs as tuples.

```
d = {"a": 1}
keys = d.keys()
print(keys)
d["b"] = 2
print(keys)
```

OUTPUT

```
dict_keys(['a'])
dict_keys(['a', 'b'])
```

The variable `keys` was not rebuilt. The view updated itself.

```
inventory = {"apple": 50, "banana": 30, "cherry": 20}
print(list(inventory.keys()))
print(list(inventory.values()))
print(list(inventory.items()))
for key in inventory:
    print(key)
for val in inventory.values():
    print(val)
for key, val in inventory.items():
    print(f"{key} costs {val}")
```

OUTPUT

```
['apple', 'banana', 'cherry']
[50, 30, 20]
[('apple', 50), ('banana', 30), ('cherry', 20)]
apple
banana
cherry
50
30
20
apple costs 50
banana costs 30
cherry costs 20
```


A plain `for key in inventory` loops over keys (the default). `.items()` is the most useful form because you get both sides at once.

Comparison: list, tuple, set, dictionary

Feature	List	Tuple	Set	Dictionary
Written with	[]	()	{ }	{key: value}
Ordered	Yes	Yes	No	Yes (Python 3.7+)
Mutable	Yes	No	Yes (items must be immutable)	Yes
Duplicates	Allowed	Allowed	Removed	Duplicate keys not allowed (last value wins)
Access by	Index 0, 1, 2...	Index 0, 1, 2...	No index	Key
Empty value	[]	()	set()	{}
Typical use	Changing sequences	Fixed records	Unique items, set maths	Look-up tables

WORKED EXAMPLE

Same data, four structures. Three subject marks:

- List `[80, 70, 90]` — easy to append a fourth test.
- Tuple `(80, 70, 90)` — a frozen result slip.
- Set `{80, 70, 90}` — unique scores only; two papers of 80 would become one 80.
- Dictionary `{"math": 80, "eng": 70, "cs": 90}` — you ask "What is CS?" and get 90 without remembering an index.

REMEMBER IT

Keys are unique, like roll numbers. Values are the student records and may repeat (two students can both score 80). Memory: "**get** is gentle, brackets can **break**."

COMMON MISTAKE

Older notes (and the chapter summary in some slides) still call dictionaries "unordered." From Python 3.7 onwards insertion order is preserved, so you may write "ordered since 3.7." Keys must be immutable: a list cannot be a key; a tuple of numbers can. Do not write `{}` when you want an empty set.

اردو میں سمجھیں

ڈکشنری (Dictionary) کلید اور قدر (Key-Value) کے جوڑوں میں ڈیٹا رکھتی ہے۔ کلید منفرد ہوتی ہے؛ دہرائی ہوئی کلید پر آخری قدر رہ جاتی ہے۔ `d[key]` غیر موجود کلید پر `KeyError` دیتا ہے، جبکہ `d.get(key, default)` محفوظ ہے۔ `.values()` اور `.keys()` سے `.items()` سے گھما جا سکتا ہے۔ فہرست انڈیکس سے، ڈکشنری کلید سے پہچانی جاتی ہے۔

EXAM TIP

High-yield: definition of a dictionary, difference between `d[key]` and `d.get(key)`, "what if a key is repeated?", traversal with `.items()`, and a full comparison table of the four structures. Memorise the comparison table in this section — it is a complete 8-mark answer if you add one example row. Always mention that `in` checks keys.

CHECK YOURSELF

1. What does `{"a": 1, "a": 2, "a": 3}` print?
2. What does `person.get("salary", 0)` return if `salary` is missing?
3. Which structure has no index and no duplicate items?

Answers: 1. `{'a': 3}` . 2. `0` . 3. A set.

3.1.5 Common Built-in Functions (min, max, len, sum)

LEARN IT

Several built-in functions work across lists, tuples, sets and dictionaries. You do not call them with a dot on the object (except when a type also has its own method, such as `list.sort()`).

Function	Purpose	Works on
<code>min()</code>	Smallest element (numeric or alphabetical)	List, tuple, set, dictionary keys
<code>max()</code>	Largest element	List, tuple, set, dictionary keys
<code>len()</code>	Number of elements (for a dict: number of keys)	List, tuple, set, dictionary
<code>sum()</code>	Adds numeric elements	List, tuple, set, dictionary values via <code>sum(d.values())</code>
<code>sorted()</code>	Returns a new sorted list ; original unchanged	List, tuple, set, dictionary keys

```
s = {5, 1, 9, 3, 7}
print(len(s))
print(min(s))
print(max(s))
print(sum(s))
print(sorted(s))
```

OUTPUT

```
5
1
9
25
[1, 3, 5, 7, 9]
```

`5 + 1 + 9 + 3 + 7 = 25` . `sorted(s)` always returns a **list**, even when the input is a set.

WORKED EXAMPLE

The same five functions on four structures.

```

nums = [8, 3, 5, 3]
t = (8, 3, 5, 3)
s = {8, 3, 5}
d = {"math": 80, "eng": 70, "cs": 90}
print(len(nums), min(nums), max(nums), sum(nums), sorted(nums))
print(len(t), min(t), max(t), sum(t), sorted(t))
print(len(s), min(s), max(s), sum(s), sorted(s))
print(len(d), min(d), max(d), sum(d.values()), sorted(d))

```

OUTPUT

```

4 3 8 19 [3, 3, 5, 8]
4 3 8 19 [3, 3, 5, 8]
3 3 8 16 [3, 5, 8]
3 cs math 240 ['cs', 'eng', 'math']

```

On the dictionary, `min(d)` and `max(d)` compare **keys as strings**: "cs" comes first alphabetically, "math" last. `sum(d)` would try to add the keys "math" + "eng" + ... and fail. Use `sum(d.values())` — 80 + 70 + 90 = 240. The set dropped the duplicate 3, so `len(s)` is 3 and `sum(s)` is 16.

sorted() versus .sort()

```

original = [8, 3, 5]
print("before sort:", original)
print("sorted():", sorted(original))
print("after sorted():", original)
original.sort()
print("after .sort():", original)

```

OUTPUT

```

before sort: [8, 3, 5]
sorted(): [3, 5, 8]
after sorted(): [8, 3, 5]
after .sort(): [3, 5, 8]

```

`.sort()` exists only on lists and rewrites the original. `sorted()` works on any iterable and leaves the original alone.

REMEMBER IT

LEN-MIN-MAX-SUM — count, smallest, largest, total. For a dictionary: `len` / `min` / `max` / `sorted` use **keys**; `sum` needs `.values()`.

COMMON MISTAKE

`sum()` needs numbers. `sum(["a", "b"])` raises `TypeError`. Empty structures: `len([])` is 0, but `min([])` and `max([])` raise `ValueError` because there is no smallest of nothing.

اردو میں سمجھیں

`len()` عناصر گنتا ہے، `min()` سب سے چھوٹا، `max()` سب سے بڑا، `sum()` اعداد کا مجموعہ، اور `sorted()` نئی ترتیب شدہ فہرست دیتا ہے بغیر اصل کو بدلے ہیں۔ ڈکشنری پر `min` اور `max` کلیدوں پر چلتے ہیں؛ قدروں کا مجموعہ `sum(d.values())` سے نکالیں۔ `sort()` صرف فہرست کا میتھڈ ہے جو اصل فہرست بدل دیتا ہے۔

CHECK YOURSELF

1. For `s = {5, 1, 9, 3, 7}`, what are `len(s)`, `min(s)`, `max(s)`, `sum(s)`?
2. Does `sorted(s)` return a set or a list?
3. How do you total the values of `{"math": 80, "eng": 70, "cs": 90}`?

Answers: 1. 5, 1, 9, 25. 2. A list: `[1, 3, 5, 7, 9]`. 3. `sum(d.values())` → 240.

3.1.6 Activity: Student Attendance System

LEARN IT

Teachers need a quick way to mark who is in class. This program stores names in a **list**, asks about each student, counts how many said yes, and prints a summary with `len()`.

Store names in a list → Ask present? for each → Count yes → Print totals

```
students = ["Ali", "Sara", "Ayesha"]
present_count = 0
print("--- Attendance System ---\n")
for student in students:
    status = input(f"Is {student} present? (yes/no): ").lower()
    if status == "yes":
        present_count += 1
print("\n--- Attendance Summary ---")
print("Total Students:", len(students))
print("Students Present:", present_count)
print("Students Absent:", len(students) - present_count)
```

WORKED EXAMPLE

Sample run. Ali yes, Sara no, Ayesha yes. `.lower()` means YES or Yes still counts as present.

OUTPUT

```
--- Attendance System ---

Is Ali present? (yes/no): yes
Is Sara present? (yes/no): no
Is Ayesha present? (yes/no): yes

--- Attendance Summary ---
Total Students: 3
Students Present: 2
Students Absent: 1
```

Student	Typed answer	After <code>.lower()</code>	<code>present_count</code>
Ali	yes	yes → add 1	1
Sara	no	not yes	1
Ayesha	yes	yes → add 1	2

Absent = `len(students) - present_count` = 3 - 2 = 1. The counter must start at 0 **before** the loop. If you put `present_count = 0` inside the loop, the count would reset for every student.

REMEMBER IT

Counter pattern: start at 0, loop, add 1 when a condition is true. You will reuse this in the word-frequency program (add 1 per word) and the grade tracker (add each mark).

COMMON MISTAKE

Only the exact string "yes" after `.lower()` counts as present. "y" or "present" is treated as absent. In a longer program you could also accept "y"; this activity keeps the rule simple.

اردو میں سمجھیں

حاضری کا پروگرام طلبہ کے نام ایک فہرست میں رکھتا ہے، ہر نام کے لیے پوچھتا ہے، اور yes پر شمار کار (Counter) ایک بڑھاتا ہے۔ کل طلبہ `len(students)` سے، غیر حاضر کل - حاضر سے نکلتے ہیں۔ شمار کار لوپ سے پہلے صفر ہونا چاہیے۔

CHECK YOURSELF

1. If all three students type no, what is printed for present and absent?
2. Why is `.lower()` used on the answer?

Answers: 1. Present 0, Absent 3. 2. So that YES, Yes and yes all match the string "yes".

3.1.7 Activity: Word Frequency Counter

LEARN IT

Count how many times each word appears in a sentence. This is a classic use of a **dictionary** plus `.get()`: the word is the key, the count is the value.

```
sentence = "python is easy python is fun python is powerful"
words = sentence.split()
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1
print(f"Sentence: '{sentence}'\n")
print("Word Frequency Table:")
for word, count in word_freq.items():
    print(f"{word:<10} : {count}")
top_word = max(word_freq, key=word_freq.get)
print(f"\nMost Frequent: '{top_word}' ({word_freq[top_word]} times)")
```

WORKED EXAMPLE

Logic, step by step.

1. `sentence.split()` cuts on spaces and gives `['python', 'is', 'easy', 'python', 'is', 'fun', 'python', 'is', 'powerful']`.
2. For a new word, `word_freq.get(word, 0)` is 0, then you store $0 + 1 = 1$.
3. For a word already in the dictionary, `.get(word, 0)` returns the current count, then you add 1.
4. `{word:<10}` pads the word to 10 characters so the colons line up.
5. `max(word_freq, key=word_freq.get)` picks the key whose value is largest. If two words share the top count, `max` returns the first of those keys in insertion order.

OUTPUT

Sentence: 'python is easy python is fun python is powerful'

Word Frequency Table:

```
python      : 3
is          : 3
easy        : 1
fun         : 1
powerful    : 1
```

Most Frequent: 'python' (3 times)

After seeing	word_freq
python	{'python': 1}
is	{'python': 1, 'is': 1}
easy	{'python': 1, 'is': 1, 'easy': 1}
python (again)	{'python': 2, 'is': 1, 'easy': 1}
... end of sentence	{'python': 3, 'is': 3, 'easy': 1, 'fun': 1, 'powerful': 1}

Here python and is both occur 3 times. max returns 'python' because it appears first in the dictionary.

REMEMBER IT

The heart of every counter is one line: `d[item] = d.get(item, 0) + 1`. You can count letters, roll numbers or votes with the same pattern.

COMMON MISTAKE

This program is case-sensitive and space-based. "Python" and "python" would be two keys. Punctuation such as `easy.` would not match `easy`. For a class exercise, keep the sentence lowercase with spaces only, as here. If you wrote `word_freq[word] += 1` without `.get` on a new word, you would get `KeyError`.

اردو میں سمجھیں

جملے کو `.split()` سے الفاظ کی فہرست بناتے ہیں، پھر ہر لفظ ڈکشنری میں کلید ہوتا ہے اور اس کی گنتی قدر۔ نئے لفظ کے لیے `get(word, 0) + 1` سے گنتی 1 سے شروع ہوتی ہے۔ `max(... , key=word_freq.get)` سب سے زیادہ بار آنے والا لفظ دیتا ہے۔

CHECK YOURSELF

1. How many times does "is" occur in the sample sentence?
2. Why use `.get(word, 0)` instead of `word_freq[word]` on the right-hand side?

Answers: 1. Three times. 2. Because a brand-new word is not yet a key; `.get` returns 0 instead of raising `KeyError`.

3.2 Functions in Python ★ Golden

LEARN IT

When the same instructions are written again and again, that is **duplicate code**. Duplicate code makes a program longer, harder to read, and painful to update (you must fix every copy). A **function** lets you write the instructions **once** and run them whenever you call the function's name.

Benefit	Meaning
Code reusability	Write once, use anywhere.
Modularity	Break a big problem into small named tasks.
Readability	Shorter, cleaner files with meaningful names.
Maintainability	Fix a bug in one place and every call is fixed.

```
def greet():  
    print("Hello, World!")  
  
greet()  
greet()
```

OUTPUT

```
Hello, World!  
Hello, World!
```

The body is written once. Two calls run it twice. To change the message you edit one line inside `greet`.

Key components of a function



- **Keyword** — Python uses `def` to start a definition.
- **Function name** — the identifier you call, e.g. `greet`.
- **Parameters** — input names inside the parentheses. When you call the function, the values you pass are called **arguments**.
- **Body** — the indented block that runs on each call.
- **Return statement** — sends a result back to the caller (3.2.2).

```
def add(a, b):  
    print(f"Adding {a} + {b}")  
    return a + b  
  
result = add(4, 6)  
print(result)
```

OUTPUT

```
Adding 4 + 6  
10
```

Here `a` and `b` are parameters. `4` and `6` are arguments. `return a + b` sends 10 back, and that 10 is stored in `result`.

WORKED EXAMPLE

Without a function you would copy the print-and-add lines every time you needed a sum. With a function, a new sum is one call: `add(10, 20)`, `add(1.5, 2.5)`.

REMEMBER IT

DRY — Don't Repeat Yourself. If you are about to copy-paste a block, wrap it in `def`. Name functions with verbs: `greet`, `add`, `save_grades`.

COMMON MISTAKE

Defining is not calling. After `def greet()`: nothing prints until you write `greet()`. Forgetting the parentheses — writing `greet` alone — does not run the body. The body must be indented; a line at column 0 is outside the function.

اردو میں سمجھیں

فنکشن (Function) ہدایات کا وہ بلاک ہے جو ایک بار لکھا جاتا ہے اور بار بار بلایا جا سکتا ہے۔ اس سے کوڈ مختصر، ماڈیولر اور آسان اصلاح والا بنتا ہے۔ تعریف `def` سے شروع ہوتی ہے، نام اور پیرامیٹرز قوسین میں ہوتے ہیں، جسم انڈینٹ ہوتا ہے، اور `return` نتیجہ واپس بھیج سکتا ہے۔ تعریف کرنے سے فنکشن چلتا نہیں جب تک آپ اسے کال نہ کریں۔

EXAM TIP

Write the four benefits (reusability, modularity, readability, maintainability) as separate bullets — that is a standard short question. For long questions, define a function, label `def`, name, parameters, body and `return`, then show one call. "What is duplicate code and how do functions solve it?" is also common.

CHECK YOURSELF

1. What keyword starts a function definition?
2. How many times does `Hello, World!` print in the `greet` program above, and why?

Answers: 1. `def`. 2. Twice, because `greet()` is called twice.

3.2.1 Types of Functions (Built-in vs User-defined)

LEARN IT

There are two main types of functions in Python.

a) Built-in functions are already defined by Python. You just call them. Examples you have already used: `len()`, `range()`, `sum()`, `print()`, `input()`, `min()`, `max()`, `sorted()`, `int()`, `float()`, `type()`.

b) User-defined functions are functions **you** create with `def` for tasks that no built-in covers in one step — greeting a user, dividing with a zero check, writing a grade file.

	Built-in	User-defined
Who writes it?	The Python language	The programmer
How you start	Just call the name	First write <code>def name(...):</code>
Examples	<code>len("Ali")</code> → 3	<code>def add(a, b): return a + b</code>
Purpose	Everyday operations needed by everyone	Your program's specific jobs

WORKED EXAMPLE

Side by side. Counting letters is built-in. Combining a greeting with a name is user-defined.

```
print(len("Sindh"))

def greet_student(name):
    print("Hello,", name)

greet_student("Hassan")
greet_student("Malika")
```

OUTPUT

```
5
Hello, Hassan
Hello, Malika
```

You did not write the code inside `len`. You did write the code inside `greet_student`, then reused it for two names.

REMEMBER IT

If Python already has it (`len`, `sum`, `max`), do not rewrite it. Write a user-defined function when you are packaging **your** steps.

COMMON MISTAKE

Do not name your function `list`, `print`, `sum` or `input`. You would hide the built-in name for the rest of that file. Choose names such as `print_bill` or `sum_marks`.

اردو میں سمجھیں

بلٹ ان فنکشن پائتھن میں پہلے سے موجود ہوتے ہیں، جیسے `len()` اور `print()`۔ یوزر ڈیفائنڈ فنکشن وہ ہیں جو پروگرامر `def` سے خود بناتا ہے تاکہ مخصوص کام بار بار کیا جا سکے۔ جو کام بلٹ ان پہلے کر دیتے ہیں انہیں دوبارہ نہ لکھیں۔

CHECK YOURSELF

1. Is `range()` built-in or user-defined?
2. Which keyword creates a user-defined function?

Answers: 1. Built-in. 2. `def`.

3.2.2 Functions with Return Values

LEARN IT

A function can either **print** a result (show it on the screen) or **return** a result (send it back to the caller). A returned value can be stored in a variable, printed later, or used in a further calculation. The call is replaced by the returned value, as if you had written that value in its place.

```
def multiply(a, b):
    return a * b

result = multiply(5, 3)
print("The answer is:", result)
```

OUTPUT

The answer is: 15

`multiply(5, 3)` evaluates to 15. That 15 is stored in `result`.

WORKED EXAMPLE

Print versus return. This is the difference examiners want.

```
def show_sum(a, b):  
    print(a + b)  
  
def give_sum(a, b):  
    return a + b  
  
x = show_sum(2, 3)  
print("stored from show_sum:", x)  
y = give_sum(2, 3)  
print("stored from give_sum:", y)  
print("double it:", y * 2)
```

OUTPUT

```
5  
stored from show_sum: None  
stored from give_sum: 5  
double it: 10
```

`show_sum` prints 5 inside the function, then returns `None` (the default when there is no `return`). You cannot double that answer later. `give_sum` hands 5 back, so `y * 2` is 10. Use `print` for messages; use `return` for results the rest of the program still needs.

REMEMBER IT

return = send back; print = show now. `return` also **exits** the function immediately. Lines after `return` in the same function do not run.

COMMON MISTAKE

Writing `print(a * b)` inside `multiply` and then `result = multiply(5, 3)` stores `None` in `result`, not 15. If you later write `result + 1` you get `TypeError`. Return the number; let the caller decide whether to print it.

اردو میں سمجھیں

`return` نتیجہ کال کرنے والے پروگرام کو واپس بھیجتا ہے تاکہ وہ متغیر میں محفوظ ہو سکے یا آگے حساب میں لگے۔ اگر `return` نہ ہو تو فنکشن `None` دیتا ہے۔ `print` صرف اسکرین پر دکھاتا ہے؛ اس سے حاصل شدہ قیمت دوبارہ استعمال نہیں ہوتی۔

CHECK YOURSELF

1. What is stored in `result` after `result = multiply(5, 3)` with `return a * b`?
2. What does a function return if it has no `return` statement?

Answers: 1. 15. 2. `None`.

3.2.3 Activity: Arithmetic Calculator Using Functions

LEARN IT

Each arithmetic operation is a **separate function**. That is modularity: addition is one small task, division is another (with a guard against dividing by zero). The main program only asks for two numbers and calls the functions.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Addition:", add(num1, num2))
print("Subtraction:", subtract(num1, num2))
print("Multiplication:", multiply(num1, num2))
print("Division:", divide(num1, num2))
```

WORKED EXAMPLE

Sample run with 12 and 4. `float` is used so decimals work; even whole numbers print with a decimal (12 becomes 12.0).

OUTPUT

```
Enter first number: 12
Enter second number: 4
Addition: 16.0
Subtraction: 8.0
Multiplication: 48.0
Division: 3.0
```

Check: $12 + 4 = 16$, $12 - 4 = 8$, $12 \times 4 = 48$, $12 \div 4 = 3$. In Python 3, `/` always produces a float, so 3.0 not 3.

Zero divisor:

```
def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

print(divide(10, 0))
print(divide(10, 2))
```

OUTPUT

```
Cannot divide by zero
5.0
```

Without the `if b != 0` test, `10 / 0` would raise `ZeroDivisionError` and stop the program.

REMEMBER IT

Four functions, four names: `add`, `subtract`, `multiply`, `divide`. The main program should look like a contents page — a sequence of calls — not a pile of formulae.

COMMON MISTAKE

`input()` returns a string. If you skip `float()` (or `int()`), `add` would concatenate: `"12" + "4"` is `"124"`. Convert first. Also, `divide` returns either a number or a string; do not later do arithmetic on the error message.

اردو میں سمجھیں

کیلکولیٹر میں ہر عمل الگ فنکشن ہے: جمع، تفریق، ضرب، تقسیم۔ یہ ماڈیولریٹی ہے۔ تقسیم سے پہلے صفر کی جانچ ضروری ہے ورنہ پروگرام رک جاتا ہے۔ `input()` سے ملنے والی تار کو `float()` سے عدد بنانا لازمی ہے ورنہ جمع جوڑنے کے بجائے الفاظ جوڑ دے گی۔

CHECK YOURSELF

1. What does `divide(10, 0)` return in this activity?
2. Why are the numbers converted with `float()` ?

Answers: 1. The string "Cannot divide by zero". 2. Because `input()` returns text; `float()` turns it into a number so `+` `-` `*` `/` do arithmetic, including decimals.

3.2.4 Scope of Local and Global Variables

LEARN IT

Scope is the region of a program where a variable can be seen and used. Python has two scopes you must know for this chapter.

- i) Local scope.** A **local variable** is created inside a function. It exists only while that function is running. The main program cannot read it.
- ii) Global scope.** A **global variable** is created outside all functions. Any function may **read** it, and the main program can use it too.

WORKED EXAMPLE

Local — the name is invisible outside.

```
def calc():
    secret = 50
    print(secret)

calc()
print(secret)
```

OUTPUT

```
50
Traceback (most recent call last):
  File "local_scope.py", line 5, in <module>
    print(secret)
NameError: name 'secret' is not defined
```

First the function prints 50. Then the main program tries `print(secret)` and fails: `secret` was born inside `calc` and died when `calc` finished.

Global — readable inside and outside.

```
x = 10

def show():
    print("Inside function:", x)

show()
print("Outside function:", x)
```

OUTPUT

```
Inside function: 10
Outside function: 10
```

Assignment inside a function creates a new local with the same name. It does not change the global:

```
x = 10

def show():
    x = 99
    print("Inside:", x)

show()
print("Outside:", x)
```

OUTPUT

```
Inside: 99
Outside: 10
```

Inside, `x = 99` is a local. After the call, the global is still 10.

REMEMBER IT

Local = inside the room; global = in the corridor. Everyone can see the corridor. Things left in a room are gone when you leave. Prefer locals. Too many globals make bugs hard to find because any function might be using them.

COMMON MISTAKE

If you only **read** `x` inside the function, Python uses the global. If you **assign** to `x` anywhere in that function, Python treats `x` as local for the whole function. Mixing the two without care causes `UnboundLocalError`. Keep function inputs as parameters and outputs as `return` values instead of editing globals.

دائرہ کار (Scope) بتاتا ہے کہ متغیر کہاں دکھائی دیتا ہے۔ لوکل متغیر فنکشن کے اندر بنتا ہے اور فنکشن ختم ہوتے ہی غائب ہو جاتا ہے، باہر سے پڑھنے پر `NameError` آتا ہے۔ گلوبل متغیر تمام فنکشنز سے باہر بنتا ہے اور اندر سے پڑھا جا سکتا ہے۔ فنکشن کے اندر اسی نام کو نئی قدر دینے سے نیا لوکل بنتا ہے، گلوبل نہیں بدلتا۔

CHECK YOURSELF

1. Where is a local variable defined?
2. In the last example, what does the main program print for `x` after `show()` ?

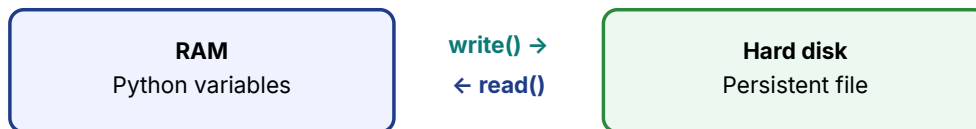
Answers: 1. Inside a function. 2. 10 (the global is unchanged).

3.3 & 3.4 File Handling and Operations (Modes `r`, `w`, `a`)

★ Golden

LEARN IT

File handling is a program's ability to create, read, write, update and delete files on disk. Variables live in **RAM** and vanish when the program ends. Files live on a **hard disk or SSD** and stay there — they are **persistent**.



Read copies data from disk into RAM. Write copies data from RAM onto disk.

Types of files

Type	What it stores	Examples
Text file	Human-readable characters. You can open it in a text editor.	<code>.txt</code> , <code>.csv</code> , <code>.py</code>
Binary file	Raw bytes, not meant to be read as text.	<code>.jpg</code> , <code>.mp3</code> , <code>.dat</code>

This chapter uses text files.

The `open()` function and modes

You must open a file before you use it: `open(filepath, mode)`.

Mode	Name	Description
<code>'r'</code>	Read	Open for reading. Fails if the file does not exist.
<code>'w'</code>	Write	Open for writing. Erases existing content. Creates the file if it is missing.
<code>'a'</code>	Append	Open for writing at the end . Keeps old content. Creates the file if it is missing.
<code>'rb'</code> / <code>'wb'</code>	Binary	Read or write bytes (images, audio).

File object methods

Method	What it does
<code>.read()</code>	Whole file as one string.
<code>.readlines()</code>	All lines as a list of strings (each usually ends with <code>\n</code>).
<code>.write("text")</code>	Writes a string. You must add <code>\n</code> yourself for a new line.
<code>.close()</code>	Saves to disk, closes the file, frees RAM. Essential if you did not use <code>with</code> .

WORKED EXAMPLE

Create a file in write mode.

```
new_file = open('myfile.txt', 'w')
new_file.write("The quick brown fox jumps over the lazy dog!!!\n")
new_file.close()
print("File has been created successfully.")
```

OUTPUT

```
File has been created successfully.
```

If `myfile.txt` did not exist, `'w'` created it. The sentence is now on disk. `.close()` flushes the buffer so the bytes actually reach the disk.

Danger of `'w'`: opening an existing file in `'w'` deletes everything first.

```
f = open('myfile.txt', 'w')
f.write("FIRST\n")
f.close()
f = open('myfile.txt', 'w')
f.write("SECOND\n")
f.close()
with open('myfile.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
SECOND
```

`FIRST` is gone. To keep old lines and add a new one, use `'a'`:

```
f = open('notes.txt', 'w')
f.write("Line 1\n")
f.close()
f = open('notes.txt', 'a')
f.write("Line 2\n")
f.close()
with open('notes.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
Line 1
Line 2
```

REMEMBER IT

r = Read existing, w = Wipe then write, a = Add at end. Remember: `write` does not add a newline unless you type `\n`.

COMMON MISTAKE

Forgetting `.close()` can leave data in a memory buffer so the file on disk looks empty or incomplete — or the file can stay locked. Section 3.6 shows `with`, which closes for you even after an error. Never use `'w'` when you only meant to add a line; that is the most common way students wipe their own data.

اردو میں سمجھیں

فائل ہینڈلنگ سے ڈیٹا ڈسک پر مستقل (Persistent) رہتا ہے، برخلاف RAM کے متغیرات کے جو پروگرام بند ہوتے ہی مٹ جاتے ہیں۔
`open(path, mode)` سے فائل کھلتی ہے: `'r'` پڑھنے کے لیے (فائل ہونی چاہیے)، `'w'` لکھنے کے لیے (پرانے مواد مٹ جاتا ہے)، `'a'` آخر میں جوڑنے کے لیے۔ کام کے بعد `.close()` ضروری ہے اگر `with` استعمال نہ ہو۔

EXAM TIP

Define file handling, contrast RAM with disk, list modes `r / w / a` with one property each, and warn that `w` overwrites. Short programs that write one line and close the file are common. Mention `.read()`, `.readlines()`, `.write()` and `.close()` by name.

CHECK YOURSELF

1. Which mode overwrites an existing file?
2. Which mode fails if the file is missing?
3. Why must `.write` often include `\n`?

Answers: 1. `'w'` . 2. `'r'` . 3. Because `write()` does not start a new line by itself; without `\n` the next write continues on the same line.

3.6 The with Statement (Context Manager) ★ Golden

LEARN IT

It is easy to forget `.close()`. If the program crashes before `.close()` runs, the file may be incomplete or corrupted. Python's `with` statement is a **context manager**: it opens the file, runs the indented block, and **always closes** the file when the block ends — even if an error occurs inside.

with open(...) as f → use f → block ends → file closed automatically

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")
# no .close() here – with already closed the file
```

After the indented block, `new_file` is closed. You do not write `new_file.close()`.

3.7 Activity: reading file content

`.read()` pulls the whole file into RAM. For a huge file that can waste memory. The professional pattern is to **iterate line by line**.


```

with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")

with open('myfile.txt', 'r') as read_file:
    for line in read_file:
        print(line, end="")
print("\n\nFile read successfully.")

```

OUTPUT

```

Context managers are safe!

File read successfully.

```

`end=""` stops `print` from adding a second newline: each line from the file already ends with `\n`.

WORKED EXAMPLE

Three ways to read the same small file.

```

with open('sample.txt', 'w') as f:
    f.write("Ali 80\nSara 90\nAyesha 75\n")

with open('sample.txt', 'r') as f:
    print("≡ .read() ≡")
    print(repr(f.read()))

with open('sample.txt', 'r') as f:
    print("≡ .readlines() ≡")
    print(f.readlines())

with open('sample.txt', 'r') as f:
    print("≡ line by line ≡")
    for line in f:
        print(line, end="")

```

OUTPUT

```

≡ .read() ≡
'Ali 80\nSara 90\nAyesha 75\n'
≡ .readlines() ≡
['Ali 80\n', 'Sara 90\n', 'Ayesha 75\n']
≡ line by line ≡
Ali 80
Sara 90
Ayesha 75

```

`repr` shows the hidden `\n` characters. `.readlines()` is handy when you need a list (the grade tracker uses this). Line-by-line looping is best for large files because only one line sits in memory at a time.

REMEMBER IT

with = “take, use, put back.” Prefer `with open(...)` as `f`: in every new program. You still choose the mode: `'r'`, `'w'` or `'a'`.

COMMON MISTAKE

You cannot read from `f` after the `with` block — the file is already closed. Do all reading and writing inside the block. Also, `print(line)` without `end=""` looks double-spaced because both the line and `print` add a newline.

with بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔ بڑی فائلیں ایک ایک سطر کے لوپ سے پڑھنا بہتر ہے بجائے پورے مواد کو .read() سے میموری میں بھرنے کے۔
 print(line, end="") دگنی خالی سطر روکتا ہے۔

EXAM TIP

Define the with statement as a context manager that guarantees close. Contrast it with manual open / close. Be ready to write a four-line program that writes with with and a four-line program that reads line by line. Mention memory: iterating is safer for large files than .read().

CHECK YOURSELF

1. What does a context manager guarantee for a file?
2. Why pass end="" when printing each line from a file?

Answers: 1. That the file is closed when the with block ends, even if an error occurs. 2. Because each line already ends with \n; print would otherwise add another newline and the output would look double-spaced.

3.8 Errors and Exceptions (Try-Except)

LEARN IT

An **exception** is an error raised while the program is running — for example opening a file that is not there. Without handling, Python prints a traceback and **stops**. A try / except block catches the error and lets you show a clear message instead of crashing.

Error type	Meaning
FileNotFoundError	The path does not exist.
PermissionError	The operating system will not let you read or write that file.
IOError	A general input/output problem (disk full, disk disconnected).

Put risky code in try. Put the recovery in except ErrorType. A final except Exception as e can catch anything else, using e as the error object.

```
try:
    with open('missing_data.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

OUTPUT

Error: The file does not exist. Please check the name.

Without `try`, the same open raises:

OUTPUT

```
Traceback (most recent call last):
  File "missing.py", line 1, in <module>
    open('missing_data.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'missing_data.txt'
```

WORKED EXAMPLE

When the file exists, the except blocks are skipped.

```
with open('ok.txt', 'w') as f:
    f.write("Present\n")

try:
    with open('ok.txt', 'r') as f:
        content = f.read()
        print(content, end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

OUTPUT

Present

Professional software does not dump a red traceback on a user. If a file is missing or a disk fails, the user should see a calm sentence such as "Please check the file name" or "Please try again later."

REMEMBER IT

try = attempt; except = backup plan. Catch the **specific** error first (`FileNotFoundError`), then a general `Exception` if you need it. Order matters: a bare `except Exception` first would swallow the specific error.

COMMON MISTAKE

Do not wrap your entire program in one silent `except:` that ignores every mistake. You would hide real bugs. Catch the errors you expect, and keep the message honest. `'w'` and `'a'` create a missing file, so they do not raise `FileNotFoundError`; `'r'` does.

اردو میں سمجھیں

ایکسپشن (Exception) چلتے پروگرام کی غلطی ہے، جیسے غیر موجود فائل کھولنا۔ `try` میں خطرناک کوڈ لکھیں اور `except` میں `FileNotFoundError` صاف پیغام دکھائیں تاکہ پروگرام کریش نہ ہو۔ پیشہ ورانہ سافٹ ویئر صارف کو سرخ ٹریس بیک کی بجائے سمجھ آنے والا جملہ دکھاتا ہے۔

CHECK YOURSELF

1. Which exception is raised when `open('missing_data.txt', 'r')` cannot find the file?
2. Does the sample program crash, or print a message, when the file is missing?

Answers: 1. `FileNotFoundError`. 2. It prints "Error: The file does not exist. Please check the name." and continues.

3.9 Mini File Handling Project: Grade Tracker

LEARN IT

This project uses 'w' to create a grades file, 'a' to add a student later, and 'r' to read every line and compute an average. It combines file handling, string methods and arithmetic.

Write names and marks ('w') → Append a new student ('a') → Read lines ('r') → Split, total, average

```
with open("grades.txt", "w") as f:
    f.write("Alice: 88\n")
    f.write("Bob: 75\n")
    f.write("Carol: 92\n")
print("Grades file created.")

with open("grades.txt", "a") as f:
    f.write("David: 81\n")
print("New student appended.")

with open("grades.txt", "r") as f:
    lines = f.readlines()
    total = 0
    for line in lines:
        name, grade = line.strip().split(":")
        print(f"{name} scored {grade}")
        total += int(grade)
    average = total / len(lines)
    print(f"\nClass average: {average:.1f}")
```

WORKED EXAMPLE

What happens in memory and on disk.

1. 'w' creates grades.txt (or wipes an old one) with three lines.
2. 'a' adds David: 81\n at the end. Alice, Bob and Carol stay.
3. .readlines() returns
['Alice: 88\n', 'Bob: 75\n', 'Carol: 92\n', 'David: 81\n'].
4. .strip() removes the trailing \n. .split(":") turns "Alice: 88" into ["Alice", " 88"].
5. int(grade) accepts the extra space: int(" 88") is 88.
6. Total = 88 + 75 + 92 + 81 = 336. Average = 336 / 4 = 84.0. The format :.1f prints one decimal place.

OUTPUT

```
Grades file created.
New student appended.
Alice scored  88
Bob scored   75
Carol scored  92
David scored  81

Class average: 84.0
```

The extra space before each mark in the output comes from the space after the colon in the file ("Alice: 88"). int() ignores it. To tidy the printout you could use grade.strip() as well; the average would not change.

REMEMBER IT

Same file, three modes, three with blocks. Never open the same file in 'w' when you only meant to add David — that would delete Alice, Bob and Carol.

COMMON MISTAKE

If a line has no colon, `split(":")` will not produce two parts and the unpacking `name, grade = ...` will raise `ValueError`. Keep the `Name: mark` format. An empty file would make `len(lines)` zero and then `total / 0` would fail — a real system should check that, just as `divide` checked for zero.

اردو میں سمجھیں

گریڈ ٹریکر پہلے 'w' سے فائل بناتا ہے، پھر 'a' سے نیا طالب علم جوڑتا ہے، پھر 'r' سے تمام سطریں پڑھ کر اوسط نکالتا ہے۔ ہر سطر کو `strip` اور `split(":")` سے نام اور نمبر میں توڑا جاتا ہے۔ اوسط = کل نمبر ÷ طلبہ کی تعداد۔ 'w' دوبارہ استعمال کرنے سے پرانے ریکارڈ مٹ جائیں گے۔

CHECK YOURSELF

1. What is the class average in the sample, and how is it calculated?
2. Why is David written with mode "a" rather than "w" ?

Answers: 1. 84.0 because $(88+75+92+81)/4 = 336/4 = 84.0$. 2. Because "a" appends; "w" would erase the three existing students.

Career Opportunities

LEARN IT

The skills in this chapter — data structures, functions and files — are the same skills used in many jobs.

- **Python / software developer** — builds applications, scripts and tools in Python, one of the most widely used languages in industry.
- **Data analyst** — uses lists and dictionaries, then file handling, to process, visualise and interpret data for decisions.
- **Automation engineer** — writes scripts that repeat file and testing tasks so humans do not have to.
- **Game developer** — uses functions, data structures and logic to build interactive games.

WORKED EXAMPLE

The word-frequency dictionary is a tiny version of what a data analyst does with a much larger file of customer comments. The grade tracker is a tiny version of a school information system. The attendance list is a tiny version of an HR register. Same structures, larger files.

REMEMBER IT

Four careers: **P-D-A-G** — **P**ython developer, **D**ata analyst, **A**utomation engineer, **G**ame developer.

COMMON MISTAKE

You do not need all four job titles for the exam. You do need one sentence each if the paper asks "List career opportunities related to this chapter."

اس باب کی مہارتیں پائٹھن ڈویلپر، ڈیٹا اینالسٹ، آٹومیشن انجینئر اور گیم ڈویلپر کے کام میں روز استعمال ہوتی ہیں۔ فہرستیں، ڈکشنریز، فنکشنز اور فائلیں چھوٹے اسکول پروگراموں سے لے کر بڑے صنعتی نظام تک ایک جیسی بنیاد ہیں۔

CHECK YOURSELF

1. Which career uses lists, dictionaries and files to interpret business data?
2. Name two careers listed in this chapter.

Answers: 1. Data analyst. 2. Any two of: Python/software developer, data analyst, automation engineer, game developer.

Chapter 3 Review

Chapter summary

- Python programs rest on primitive types (`int`, `float`, `bool`) and abstract structures: lists, tuples, sets and dictionaries.
- A **list** is ordered and mutable, written with `[]`. Index from 0; `[-1]` is last. Slice stop is exclusive. Methods include `append`, `insert`, `remove`, `sort`, `reverse`, `clear`.
- A **tuple** is ordered and immutable, written with `()`. Item assignment raises `TypeError`. Only `count` and `index` exist as methods.
- A **set** is unordered and unique, written with `{}`. Use `|` & `-` ^ for union, intersection, difference and symmetric difference. Prefer `discard` over `remove`.
- A **dictionary** maps unique keys to values. `d[key]` can raise `KeyError`; `d.get(key, default)` is safe. Duplicate keys keep the last value.
- `min`, `max`, `len`, `sum` and `sorted` work across structures. On a dict, `min/max/sorted` use keys; total values with `sum(d.values())`. `sorted()` returns a new list; `.sort()` changes a list in place.
- Functions written with `def` give reusability, modularity, readability and maintainability. Built-in functions already exist; user-defined functions solve your tasks.
- `return` sends a value to the caller. Without `return`, the result is `None`. Local variables live inside a function; global variables live outside all functions.
- Files persist on disk. Modes: `r` read (must exist), `w` write (overwrites), `a` append. Methods: `read`, `readlines`, `write`, `close`.
- The `with` statement is a context manager that closes the file automatically. Loop line by line to read large files safely.
- `try/except FileNotFoundError` stops a missing file from crashing the program.
- The grade tracker writes with `w`, appends with `a`, reads with `r`, splits each `Name: mark` line, and computes the mean.

Key terms

Term	Meaning	اردو
Data structure	A way of organising values so a program can store and use them.	ڈیٹا سٹرکچر
Primitive type	A basic single value: <code>int</code> , <code>float</code> , <code>bool</code> .	بنیادی قسم
List	Ordered, mutable sequence written with <code>[]</code> .	فہرست
Index	The position of an item, starting at 0 (or -1 from the end).	انڈیکس
Slicing	Copying a portion of a sequence with <code>start:stop:step</code> .	سلائسنگ
Traversal	Visiting each element in turn, usually with a loop.	گشت / تراورسل
Mutable	Can be changed after creation.	قابل تبدیلی
Immutable	Cannot be changed after creation.	ناقابل تبدیلی
Tuple	Ordered, immutable sequence written with <code>()</code> .	ٹیپل
Set	Unordered collection of unique items written with <code>{}</code> .	سیٹ
Union	All unique items from two sets (<code> </code>).	اجتماع
Intersection	Items common to both sets (<code>&</code>).	اشتراک
Dictionary	Mutable map of unique keys to values.	ڈکشنری
Key	The unique lookup name in a dictionary.	کلید
Value	The data stored under a key.	قدر
Function	A named, reusable block of code.	فنکشن
Parameter	An input name in a function definition.	پیرامیٹر
Return value	The result sent back with <code>return</code> .	واپسی کی قدر
Local variable	A name that exists only inside a function.	مقامی متغیر
Global variable	A name defined outside all functions.	عالمی متغیر
File handling	Creating, reading, writing and closing disk files.	فائل ہینڈلنگ
File mode	How a file is opened: <code>r</code> , <code>w</code> , <code>a</code> (or binary).	فائل موڈ
Context manager	A <code>with</code> block that sets up a resource and cleans it up.	کانٹیکسٹ مینیجر
Exception	A runtime error object, such as <code>FileNotFoundError</code> .	ایکسپشن
try / except	A pair of blocks that attempt code and handle errors.	ٹرائی / ایکسپٹ

Practice: multiple choice

- Which structure is ordered and mutable?
 - List
 - Tuple
 - Set
 - Frozen set
- What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
 - `[2, 3, 4, 5, 6]`
 - `[2, 3, 4, 5, 6, 7]`
 - `[2, 3, 4, 5]`
 - `[3, 4, 5, 6]`
- `t = (1, 2, 3)` then `t[0] = 99` raises
 - `KeyError`
 - `TypeError`
 - `ValueError`
 - `IndexError`
- A set is best described as
 - ordered with duplicates
 - indexed and mutable
 - key-value pairs
 - unordered with unique elements
- If `A = {1, 2, 3, 4}` and `B = {3, 4, 5, 6}`, then `A & B` is
 - `{3, 4}`
 - `{1, 2}`
 - `{1, 2, 3, 4, 5, 6}`
 - `{1, 2, 5, 6}`
- `{"a": 1, "a": 2, "a": 3}` prints
 - `{'a': 1}`
 - `{'a': 1, 'a': 2, 'a': 3}`
 - `{'a': 3}`
 - a `KeyError`
- The safe way to read a possibly missing dictionary key is
 - `d[key]`
 - `d.get(key)`
 - `d.keys(key)`
 - `d.index(key)`
- For `s = {5, 1, 9, 3, 7}`, `sum(s)` is
 - 5
 - 9
 - 7
 - 25
- Which statement is true?
 - `sorted(x)` returns a new list and leaves `x` unchanged
 - `sorted(x)` works only on lists
 - `.sort()` works on tuples
 - `.sort()` returns a new list
- A function is created with the keyword
 - `func`
 - `def`
 - `define`
 - `return`
- A function that has no `return` statement returns
 - 0
 - False
 - None
 - an empty string
- A variable defined inside a function is
 - local
 - global
 - persistent
 - public
- Opening a file with mode `'w'`
 - fails if the file already exists
 - adds new text at the end
 - is read-only
 - erases existing content (and creates the file if needed)
- The `with open(...) as f:` statement is used mainly because
 - it makes the file binary
 - it closes the file automatically
 - it is faster than lists
 - it prevents `KeyError`
- Reading a missing file in mode `'r'` without `try` raises
 - `TypeError`
 - `KeyError`
 - `FileNotFoundError`
 - `ValueError`

Practice: short questions

1. Define a data structure. Distinguish primitive from abstract data structures with one example of each.
2. What is negative indexing? Give the expression that returns the last item of a list named `fruits`.
3. State four differences between a list and a tuple.
4. Differentiate `.remove()` and `.discard()` on a set.
5. Why can a dictionary not have two identical keys? What happens if you write the same key twice?
6. Contrast `sorted()` with list `.sort()`.
7. List four benefits of using functions.
8. Define local scope and global scope.
9. Write the meaning of file modes `'r'`, `'w'` and `'a'`.
10. What is a context manager? Why is `with` safer than calling `.close()` yourself?

Practice: long / problem questions

1. Compare list, tuple, set and dictionary in a table (syntax, order, mutability, duplicates, how you access an item, one typical use). Then write two lines of Python that create one example of each holding the numbers 1, 2 and 2, and state what each structure actually stores.
2. Write a complete Python program that takes a sentence, counts the frequency of each word using a dictionary and `.get()`, prints a two-column table, and prints the most frequent word. Trace your program on `"python is easy python is fun python is powerful"`.
3. Write an arithmetic calculator with four user-defined functions `add`, `subtract`, `multiply` and `divide`. `divide` must not crash when the second number is 0. Explain how `return` is used and why `input` values are converted with `float`.
4. Explain file handling. Describe modes `r`, `w` and `a`, the `with` statement, and `try/except FileNotFoundError`. Include a short program that writes one line with `with` and another that reads a file that might be missing.
5. The grade tracker writes three students with `'w'`, appends David with `'a'`, then computes the class average. Write the full program, show the exact output for marks 88, 75, 92 and 81, and explain `strip`, `split(":")` and `average:.1f`.

Answer key

Multiple choice

1. a
2. c
3. b
4. d
5. a
6. c
7. b
8. d
9. a
10. b
11. c
12. a
13. d
14. b
15. c

Short questions — model answers

1. A data structure organises values in a program. Primitive types hold one value (e.g. `int 17`). Abstract structures hold collections and richer operations (e.g. a list of marks).
2. Negative indexing counts from the right. `fruits[-1]` is the last item. `fruits[-2]` is the second last.
3. List: `[]`, mutable, many methods, slightly slower, use when data will change. Tuple: `()`, immutable, only `count / index`, slightly faster, use for protected records. Both are ordered and allow duplicates and indexing.
4. Both delete an element. `remove` raises `KeyError` if the value is absent. `discard` does nothing in that case, so it is the safer choice.
5. Each key must identify one value (like one roll number per student). A repeated key does not crash; the last assigned value overwrites the earlier one, e.g. `{"a": 3}`.
6. `sorted(x)` is a built-in function for any iterable; it returns a new list and does not change `x`. `.sort()` is a list method that sorts in place and returns `None`.
7. Reusability (write once, call many times), modularity (small named tasks), readability (shorter files), maintainability (fix one definition).
8. Local: defined inside a function; invisible outside; gone when the function ends. Global: defined outside all functions; readable throughout the program.
9. `'r'` read, error if missing. `'w'` write, create if missing, erase old content. `'a'` append at the end, create if missing, keep old content.
10. A context manager sets up a resource and cleans it up. `with open(...)` as `f` closes the file when the block ends, even after an exception, so you cannot forget `.close()` and you are safer if the program crashes mid-write.

Long questions — model answers

1. Use the comparison table from 3.1.4 (brackets, ordered/mutable/duplicates/access/empty/use). Examples:

```
print([1, 2, 2])
print((1, 2, 2))
print({1, 2, 2})
print({"a": 1, "b": 2, "b": 2})
```

OUTPUT

```
[1, 2, 2]
(1, 2, 2)
{1, 2}
{'a': 1, 'b': 2}
```

The list and tuple keep both 2s and an order. The set drops the extra 2. The dictionary keeps unique keys; the second "b" overwrites the first.

2. Use the program in 3.1.7. Trace: nine tokens; `python` and `is` end at 3; others at 1. Output matches the worked example, including Most Frequent: 'python' (3 times) because `max` returns the first key among ties. Heart of the loop: `word_freq[word] = word_freq.get(word, 0) + 1`.
3. Use the four functions from 3.2.3. `return` sends each result to `print` in the main program. `divide` returns the string "Cannot divide by zero" when `b == 0`. `float(input(...))` is required because `input` returns a string and `"12" + "4"` would concatenate. Sample 12 and 4: 16.0, 8.0, 48.0, 3.0.
4. File handling stores data on disk so it outlives the program. Modes: `r` read / must exist; `w` wipe-or-create; `a` add-or-create. `with` closes automatically. Try/except:

```
with open("note.txt", "w") as f:
    f.write("Saved\n")
try:
    with open("note.txt", "r") as f:
        print(f.read(), end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

OUTPUT

```
Saved
```

If the name were wrong, the except message would print instead of a traceback.

5. Full program as in 3.9. Disk after append:

OUTPUT

```
Alice: 88
Bob: 75
Carol: 92
David: 81
```

Output of the program (exact):

OUTPUT

```
Grades file created.
New student appended.
Alice scored 88
Bob scored 75
Carol scored 92
David scored 81

Class average: 84.0
```

`strip` removes `\n`; `split(":")` separates name and mark; `int(grade)` allows the extra space; `{average:.1f}` prints one decimal. $336 \div 4 = 84.0$.

Self-assessment: I can...

- ☐ Choose list, tuple, set or dictionary for a given problem and justify the choice.
- ☐ Explain tuple immutability and the inner-list exception, and use `count / index`.
- ☐ Update a dictionary, access values with `.get()`, and loop with `.items()`.
- ☐ Write a user-defined function with parameters and a `return` value, and contrast it with a built-in.
- ☐ Open a text file in `r`, `w` or `a` using `with`, and read it line by line.
- ☐ Index, slice, traverse and update a list, including `[-1]` and the main methods.
- ☐ Build a set, drop duplicates, and compute union, intersection, difference and symmetric difference.
- ☐ Apply `min`, `max`, `len`, `sum` and `sorted` correctly, including on dictionaries.
- ☐ State whether a name is local or global and predict a `NameError`.
- ☐ Catch `FileNotFoundError` and complete the grade-tracker project with the correct average.